

Product Requirements Document (PRD)

Multi-Tenant SaaS API — product vision, goals, users, requirements, journeys, risks and success criteria

1. Executive Summary

The Multi-Tenant SaaS API is a project and task management platform centered around organizations. Users belong to organizations, organizations contain projects, projects contain tasks, and users collaborate through comments and notifications. The initial objective is engineering-first: demonstrate production-grade backend design while preserving a path to future commercialization.

2. Vision

Build a secure, scalable and observable multi-tenant SaaS backend that can support organizations in managing projects, tasks and collaboration, while serving as a strong production-style backend engineering project.

3. Goals

- Secure multi-tenant architecture with strong tenant isolation.
- Robust authentication using JWT access tokens and refresh tokens.
- Organization and project-level authorization.
- Full project/task management rather than a minimal CRUD task manager.
- Architecture that can evolve toward approximately 100K users.
- Production-grade Redis, Kafka, observability, testing, Docker and CI/CD capabilities.
- A modular architecture that can later support commercialization without forcing premature microservices.

4. Non-Goals

- 2FA, OAuth and enterprise SSO in the initial scope.
- Actual file upload and object storage in the initial scope.
- Billing, subscriptions and payments.
- Advanced multiple-membership models.
- Mobile applications.
- Real-time collaboration.
- Microservices as the initial architecture.
- Elasticsearch/OpenSearch unless future search requirements justify it.

5. Personas and Roles

Role	Primary responsibilities
Organization Owner	Owns the organization and has the highest organization-level authority.
Organization Admin	Manages organization members and permitted organization settings.

Role	Primary responsibilities
Project Manager	Manages project-level work and project permissions.
Member	Performs work and interacts with projects/tasks according to permissions.
Viewer	Has read-oriented access where permitted.

6. Product Hierarchy

User → Organization → Project → Task → Comment

Users can belong to organizations. Projects belong to an organization. Tasks belong to projects. Comments are associated with tasks. File metadata is associated with relevant resources in the future, without actual upload/storage in the initial release.

7. Core User Journey

Register → Verify email → Login → Create organization → Invite members → Assign roles → Create project → Add project members → Create tasks → Assign tasks → Work on tasks → Comment/collaborate → Receive notifications → Track progress.

8. Authentication and Authorization

- Registration and login use email/password initially.
- JWT access tokens protect API requests.
- Refresh tokens support access-token renewal.
- Sessions/refresh-token state can be managed using Redis.
- Logout revokes/invalidate the relevant session.
- Authorization is evaluated using user, organization, project, role, permission and resource context.
- Tenant isolation is mandatory for every organization-scoped resource.

9. Task Model

Attribute	Initial support
Title	Required task title.
Description	Rich task description as supported by the API contract.
Status	BACKLOG, TODO, IN_PROGRESS, IN_REVIEW, DONE, CANCELLED.
Priority	LOW, MEDIUM, HIGH, URGENT.
Assignee	User responsible for the task.
Creator	User who created the task.
Due date	Optional deadline.
Labels	Categorization/filtering support.
Project	Owning project.
Comments	Collaboration associated with the task.

10. Search, Filtering, Sorting and Pagination

These are treated as reusable API query capabilities rather than isolated features. PostgreSQL is the initial search engine. Elasticsearch/OpenSearch is deferred until scale or search requirements justify additional

infrastructure.

11. Notifications and Background Processing

Domain events should be emitted asynchronously. Kafka is the preferred event backbone. Potential topics include `user.events`, `organization.events`, `project.events`, `task.events`, `comment.events` and `notification.events`. Consumers can process notification delivery, task-related work and maintenance tasks.

Example flow: Task Assigned → `task.assigned` event → notification consumer → email delivery.

12. Redis Responsibilities

- Rate limiting.
- Caching.
- Session/token management.
- Distributed locks where a critical operation requires single-writer coordination.

13. Observability

- Structured logs.
- Request/correlation IDs.
- Prometheus metrics.
- Grafana dashboards.
- OpenTelemetry distributed tracing.
- Health and readiness endpoints.
- Metrics for request count, error rate, latency, dependency latency, Kafka consumer lag, job failures, authentication failures and rate-limit rejections.

14. Success Criteria

- A user can securely authenticate and manage sessions.
- A user can create and operate within an organization.
- Organization and project permissions are enforced consistently.
- Cross-tenant access attempts fail safely.
- Projects, tasks and comments support complete core workflows.
- Search/query capabilities work consistently across collection endpoints.
- Asynchronous notification processing works without blocking the primary request.
- Operators can diagnose failures using logs, metrics and traces.
- Automated tests cover important business and integration paths.
- The system can be containerized and validated through CI/CD.

15. Risks and Mitigations

Risk	Severity	Mitigation
Tenant isolation failure	Critical	Centralize tenant/authorization checks; integration-test cross-tenant access.
Over-engineering	High	Use a modular monolith; defer microservices and commercial features.
Distributed-system complexity	High	Introduce Redis/Kafka incrementally after core domain workflows are stable.
Inconsistent authorization	High	Define a permission matrix and enforce policy at service/domain boundaries.
Unobservable failures	Medium	Make structured logs, metrics, traces and correlation IDs part of the normal implementation.
Large feature surface	Medium	Use P0/P1/P2/P3 priorities and phase gates.

16. MVP Boundary

The first usable MVP ends after organization-based multi-tenancy, RBAC, projects, tasks, comments, authentication, tenant isolation, API conventions and OpenAPI are working together. Production capabilities such as Redis, Kafka, notifications and advanced observability follow as subsequent phases rather than blocking the first end-to-end domain workflow.